VEXON DEVELOPER HANDBOOK · GUIDE 03

Node.js and Express Backend Guide

How the VEXON API in server/ starts, routes, validates, authorises, stores data and reports errors

Written for	A student who wants to read, debug and extend VEXON's backend without breaking its security rules
Project	VEXON / CampusConnect (repository: this folder's parent project)
Evidence	Generated from a direct inspection of the repository source code
Repository state	branch fix/csrf-legacy-cookie, commit af623fb, plus 126 uncommitted working-tree change(s)
Generated	16 October 2026, 21:11
Rebuild	python docs/developer-handbook/build_handbook.py

Accuracy rule: every statement about VEXON in this guide is based on files in the repository. Anything that could not be confirmed is labelled 'unverified', 'planned' or 'hypothetical'. No secrets, passwords or real student data are reproduced.

Contents

The backend in one paragraph . . . 3
Node.js and Express fundamentals . . . 3
Startup and configuration . . . 3
The middleware chain (app.js) . . . 4
Routes, middleware, controllers, services and models . . . 4
The router file . . . 5
A complete request, line by line: raising an SOS . . . 5
HTTP methods, status codes and JSON . . . 6
Authentication and authorization in the backend . . . 7
Input validation and error handling . . . 7
Two validation layers . . . 7
How errors flow . . . 8
Async operations and database access . . . 8
File uploads . . . 9
External integrations . . . 9
Socket.IO and real-time events . . . 9
Logging and troubleshooting . . . 10
How to trace a failed request . . . 10
How to debug a backend exception . . . 10
How to add a small API endpoint safely . . . 11
Practice exercises . . . 11
Source references . . . 12

The backend in one paragraph

`server/` is a **Node.js** program (ES modules, Node 18.18 or newer; this machine runs Node 24). It uses **Express 4** for HTTP, **Mongoose 8** for MongoDB and **Socket.IO 4** for real-time events. One process serves the REST API under `/api`, the WebSocket endpoint `/socket.io`, uploaded files under `/uploads`, and, when `client/dist` exists, the built web app. Everything is started from `server/src/server.js`.

Node.js and Express fundamentals

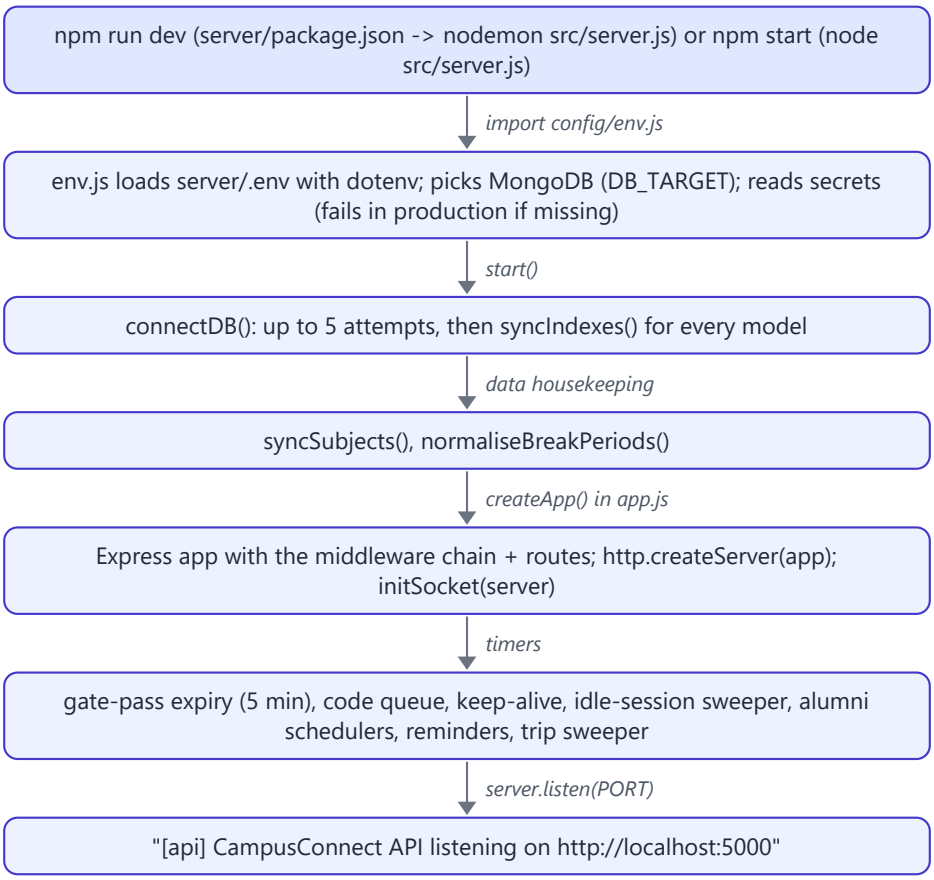
Node.js runs JavaScript outside the browser. It is *single-threaded and event-driven*: while one request waits for MongoDB, Node serves other requests. This is why every database call is `await`-ed. Code that blocks (a giant loop, synchronous file reads in a request) slows down *every* user.

Express is a small framework built on a pipeline of **middleware**. Each middleware receives `(req, res, next)` and either:

- **responds** (`res.json(...)`, `res.status(404).json(...)`), which ends the pipeline, or
- **passes control on** with `next()`, or
- **reports an error** with `next(err)`. Express then skips to the error-handling middleware.

A **route** is middleware attached to one method and path: `router.post('/sos', ...handlers)`.

Startup and configuration



Configuration rules in `server/src/config/env.js`:

- `secret(name, devDefault)`: in **development** a missing secret prints `[env] ... using an insecure development default` and the server still starts. In **production** (`NODE_ENV=production`) a missing

`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` or `CHAT_ENCRYPTION_KEY` **throws**, so the server refuses to start. This is deliberate.

- `num(name, fallback, {min, max})` reads numeric settings and ignores out-of-range values.
- The full list of variable names is in `server/.env.example`. Guide 08 groups them. Never copy real values into documents or commits.

The middleware chain (app.js)

`createApp()` in `server/src/app.js` registers middleware in this order. Order matters: a request passes through them top to bottom.

#	Middleware	Purpose
1	`helmet({...})`	Security headers, including a Content-Security-Policy that only allows map origins from `MAP_STYLE_URL` / `MAP_CSP_ORIGINS`; HSTS in production
2	Permissions-Policy header	`camera=(self), microphone=(), geolocation=(self)`
3	`cors({ origin: env.clientUrls, credentials: true })`	Only origins listed in `CLIENT_URL` may call the API from a browser
4	`compression()`	gzip responses
5	`express.json({ limit: '200kb' })`	Parse JSON bodies; keeps the raw body for the tracker webhook's HMAC check
6	`cookieParser()`	Read the refresh and CSRF cookies
7	`mongoSanitize()`	Strip `$` and `.` keys (blocks NoSQL operator injection)
8	`inputHygieneGuard`	Null-byte stripping, HTTP parameter pollution defence, pagination cap
9	`morgan('dev')`	Request log lines in development only
10	`/uploads` static + DB-stored files	Serve uploaded files with `nosniff` and a strict CSP
11	`Cache-Control: no-store` on `/api`	API responses are never cached
12	`GET /api/health`	Health check used by Render (`render.yaml`)
13	`apiLimiter`, `csrfAndOriginGuard`, `routes`	Global rate limit (1000 per 15 min), CSRF and origin check, then the router
14	`notFound`, static `client/dist`, SPA fallback, `errorHandler`	404s, the built web app, and the central error handler

Routes, middleware, controllers, services and models

VEXON uses a **layered** structure. Knowing which layer owns which job tells you where to look.

Layer	Folder	Job	Example
Routes	`server/src/routes/`	URL + method; middleware chain; validation rules	`router.post('/sos', writeLimiter, body('type')..., validate, sos.createAlert)`
Middleware	`server/src/middleware/`	Cross-cutting checks: auth, roles, CSRF, rate limits, validation, errors	`protect`, `authorize`, `validate`, `errorHandler`
Controllers	`server/src/controllers/`	Business rules for one request; read and write models; respond	`sosController.createAlert`

Layer	Folder	Job	Example
Services	`server/src/services/`	Bigger reusable logic used by several controllers or timers	`services/transport/ingest.js`, `services/attendanceReport.js`, `services/ai/studyAssistantService.js`
Models	`server/src/models/`	Data shape, validation, indexes	`SOSAlert.js`, `User.js`
Utils	`server/src/utils/`	Small helpers	`http.js` (`ApiError`, `asyncHandler`, `pick`), `notify.js`, `storage.js`

The router file

`server/src/routes/index.js` (about 2,000 lines) holds **every** route and its validation, in module sections (`// -- Auth --`, `// -- Campus SOS --`, ...). Three lines near the top divide public from private routes:

```
js
// Everything below requires a signed-in user.
router.use(protect);
// COE (Timetable / Exam Timetable / Attendance) and Office (attendance, read)
// are confined to their own dashboards — deny-by-default.
router.use(confinedRoleGuard);
// Live "something changed" signals for announcements / events / clubs / discussions.
router.use(realtimeChanges);
```

Only routes registered **above** these lines are public: register, login, refresh, logout, the retired password reset, the alumni claim and the HMAC-signed tracker webhook. Alumni routes are mounted from `server/src/routes/alumniRoutes.js` with `router.use('/alumni', alumniRouter)`.

> **Warning**
> If you ever add a route **above** `router.use(protect)`, it becomes reachable without signing in. Keep new routes below that line unless they are truly public, and then add a test that proves what an anonymous caller can and cannot do.

A complete request, line by line: raising an SOS

The route, from `server/src/routes/index.js`:

```
js
router.post(
  '/sos',
  writeLimiter,
  body('type').isString().isIn(SOS_TYPES).withMessage('Valid SOS type is required'),
  body('note').optional().isString().trim().isLength({ max: 300 }),
  body('location').optional().isObject(),
  body('location.lat').optional().isFloat({ min: -90, max: 90 }).toFloat(),
  body('location.lng').optional().isFloat({ min: -180, max: 180 }).toFloat(),
  body('location.accuracy').optional().isFloat({ min: 0 }).toFloat(),
  validate,
  sos.createAlert
);
```

- It is below `router.use(protect)`, so a valid session is required. There is **no** `authorize(...)` here on purpose: anyone signed in can raise an SOS.
- `writeLimiter` allows 40 writes per minute per IP (`server/src/middleware/rateLimit.js`).
- Each `body(...)` line is an **express-validator** rule. `SOS_TYPES` comes from `server/src/constants.js` (`medical`, `security`, `fire`, `ragging`, `other`).

- `validate` (in `server/src/middleware/validate.js`) collects the failures and calls `next(new ApiError(422, firstMessage, errors))`.

The controller, from `server/src/controllers/sosController.js`:

```
js
export const createAlert = asyncHandler(async (req, res) => {
  const data = pick(req.body, ['type', 'note', 'location']);
  data.user = req.user._id;
  data.department = req.user.department;
  data.section = req.user.section;
  data.status = 'active';

  const recent = await SOSAlert.countDocuments({ user: req.user._id, createdAt: { $gte: new
    Date(Date.now() - SOS_BURST_WINDOW_MS) } });
  if (recent >= SOS_BURST_LIMIT) {
    throw new ApiError(429, 'You have raised several alerts in the last few minutes. ...');
  }

  const alert = await SOSAlert.create(data);
  logActivity(req, 'sos.create', { entityType: 'sos', entityId: alert._id, summary: data.type });
  ...
  await notifyRoles(SOS_BROADCAST_ROLES, { type: 'sos', title: ..., message: ..., link: '/sos' }, {
    exclude: req.user._id });
  emitToRoles(SOS_RESPONDER_ROLES, 'sos:new', payload);
  emitToRoles(SOS_BYSTANDER_ROLES, 'sos:new', bystanderView(payload));
  res.status(201).json(populated);
});
```

Line by line:

1. `asyncHandler(...)` (from `server/src/utils/http.js`) catches any rejected promise and forwards it to `errorHandler`. Without it, an `await` that throws would crash or hang the request.
2. `pick(req.body, [...])` copies only the allowed fields. The client cannot set `status` or `user`.
3. `user`, `department` and `section` come from `req.user`, which `protect` loaded **from the database**, not from the request body.
4. A burst limit of 5 alerts per 10 minutes returns `429` with a helpful message.
5. `SOSAlert.create(data)` validates against the schema and inserts the document.
6. `logActivity` writes an audit row and never throws (`server/src/utils/activity.js`).
7. `notifyRoles` stores notifications for every active user in those roles and pushes them live.
8. Two different socket payloads: **responders** get the full alert (with location and phone), while **bystanders** get `bystanderView` without private fields. This is privacy by design.
9. `201 Created` with the saved alert.

HTTP methods, status codes and JSON

Method	Meaning in VEXON	Example
GET	Read; never changes data	`GET /api/notifications?page=2`
POST	Create, or perform an action	`POST /api/complaints`, `POST /api/auth/login`
PUT	Replace or update a whole resource	`PUT /api/events/:id`
PATCH	Partial update or state change	`PATCH /api/gate-pass/:id/hod-review`
DELETE	Remove	`DELETE /api/notifications/:id`

Status codes you will see, and where they come from:

Code	Meaning	Typical VEXON source
200 / 201	OK / Created	`res.json(...)` / `res.status(201).json(...)`
400	Bad request, e.g. a malformed ObjectId	`errorHandler` turns Mongoose `CastError` into 400
401	Not signed in, or the session ended	`authenticateAccessToken` in `middleware/auth.js`
403	Signed in but not allowed; CSRF failed	`authorize(...)`, `confinedRoleGuard`, `csrfAndOriginGuard`
404	No such route or record	`notFound` in `middleware/error.js`; `ApiError(404, ...)` in controllers
409	Conflict, e.g. a duplicate unique key	Mongo error `11000` becomes 409 "That ... is already in use"
410	Gone	Retired email password-reset endpoints
413	Body or upload too large	JSON over 200 kb; Multer `LIMIT_FILE_SIZE`
422	Validation failed	`validate` middleware; Mongoose `ValidationError`
423	Account locked	`authController.login`
429	Too many requests	`express-rate-limit` limiters; SOS burst limit
500	Unexpected server error	Anything not handled; the message is masked in production

Every error response has the same JSON shape: `{ "message": "...", "errors"?: [{ "field", "message" }], ...extra }`. The web client's `errMsg()` and the Kotlin `errMsg()` both read `message`.

Authentication and authorization in the backend

The full design is in Guide 01. For backend work, remember these entry points:

- `protect` reads `Authorization: Bearer <token>`, calls `authenticateAccessToken`, and sets `req.user`, `req.sessionId` and `req.sessionIdleTimeoutMs`.
- `authorize('admin', 'principal')` returns 403 unless `req.user.role` is in the list.
- **Scope** (department, class, ownership) is checked *inside* controllers. Examples: `assertCanWrite(user, cls)` in `attendanceController.js`, the HOD department checks in `timetableController.js`, and the helpers in `server/src/utils/academicScope.js`, `peopleScope.js`, `chatScope.js` and `studyMaterialScope.js`.
- Socket connections use the **same** `authenticateAccessToken` (see `server/src/config/socket.js`).

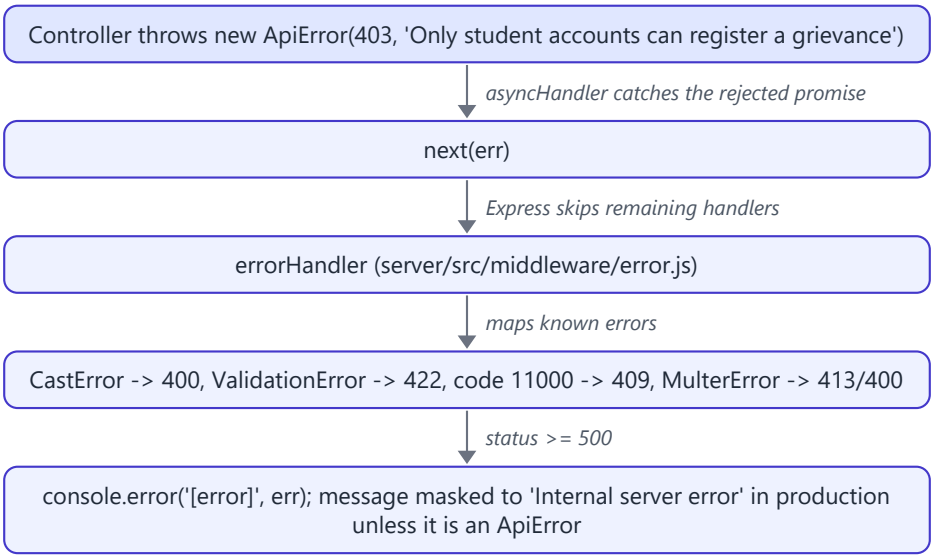
> **Key idea**
> When you add a feature, ask three questions: *who* may call it (`authorize`), *on which records* (a scope check in the controller), and *which fields* may they set (`pick`). Missing any one of these is a security bug.

Input validation and error handling

Two validation layers

1. **Request rules** (express-validator) in `routes/index.js` check type, length, allowed values and ids (`idParam('id')` rejects anything that is not a MongoDB ObjectId).
2. **Schema rules** (Mongoose) in `models/*.js` check `required`, `enum`, `min`/`max`, `match`, and custom `pre('validate')` hooks. For example, `User.js` checks that the Class In-Charge year matches the semester.

How errors flow



`ApiError(status, message, errors, data)` lives in `server/src/utils/http.js`. Use it for every *expected* failure. Anything else that is thrown becomes a 500.

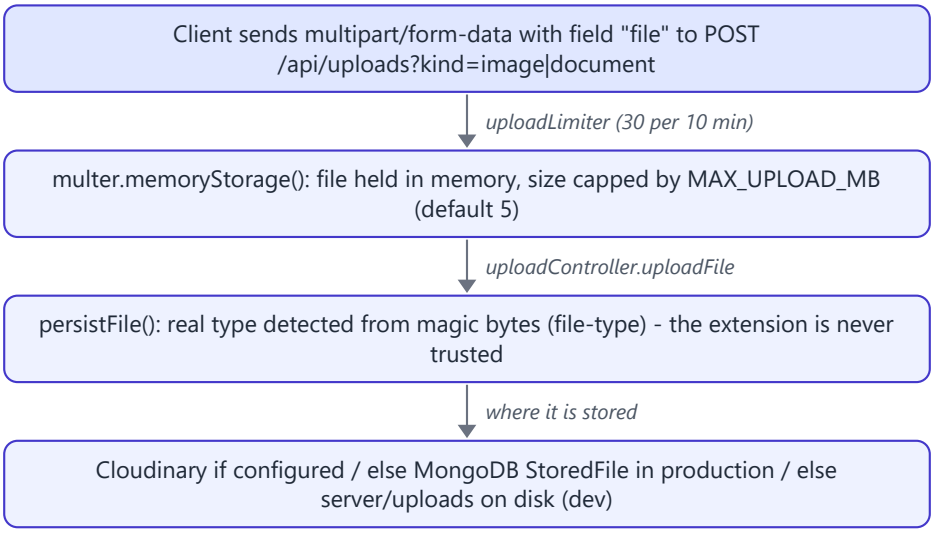
> **Common mistake**
> Writing `res.status(400).send('bad')` in one place and `throw new ApiError(...)` in another. Clients expect JSON with `message`. Always throw `ApiError` so the response shape stays consistent.

Async operations and database access

- Controllers `await` Mongoose calls: `findById`, `find().select().lean()`, `countDocuments`, `create`, `updateOne`, `aggregate([...])`.
- `.lean()` returns plain objects (faster, read-only). Use it for reads you will not `save()`.
- `.select('name email')` limits fields. `User.js` hides secrets with `select: false`, and its `toJSON` deletes the password, token version and similar fields as well.
- Parallel reads use `Promise.all([...])`, as in `authenticateAccessToken` (user + session at once).
- **Transactions**: `withTransaction(fn)` in `server/src/utils/transaction.js` runs writes atomically on a replica set (Atlas). On a standalone local `mongod` it runs them without a transaction. Attendance and exam code use it.

Guide 04 covers queries, indexes and relationships in depth.

File uploads



Files are served with `X-Content-Type-Options: nosniff` and a restrictive CSP (`app.js`). `readStoredFile` accepts only a generated `uuid.ext` style name, which blocks path traversal.

External integrations

Integration	Code	Behaviour when not configured
Groq LLM (Study Assistant)	`server/src/services/ai/groqService.js`, `studyAssistantService.js`, `pdfIndexer.js`	The assistant is unavailable; the rest of the app works
College SMS (parent OTP, exit notice)	`server/src/services/otp/otpService.js`, `providers/collegeSmsProvider.js`, `providers/testOtpProvider.js`	With `OTP_SMS_ENABLED=false`, OTPs are generated and verified but not sent; admins see them in Security & Logs
Push	`server/src/services/pushService.js`, `fcm.js`	No push; in-app notifications and sockets still work
Cloudinary	`server/src/utils/storage.js`	Falls back to MongoDB or disk
Email (nodemailer)	`server/src/utils/mailer.js`	Logs a warning; nothing is sent
Vehicle GPS tracker	`transportController.trackerWebhook`, `services/transport/ingest.js`	The webhook is disabled until `TRACKER_WEBHOOK_SECRET` is set

The Study Assistant streams answers with **Server-Sent Events** when the request has `stream: true`. See `server/src/controllers/aiController.js`: it sets `Content-Type: text/event-stream` and writes `event: delta` lines, then `done` or `error`.

Socket.IO and real-time events

Emit helpers in `server/src/config/socket.js`:

Helper	Sends to
`emitTo(room, event, payload)`	One room, e.g. `user:<id>` or `chat:<id>`
`emitToRooms(rooms, ...)`	Several rooms; each socket receives the event once
`emitToUsers(userIds, ...)`	Each user's `user:<id>` room
`emitToRoles(roles, ...)`	`role:<role>` rooms
`broadcast(event, payload)`	Everyone (used by `middleware/realtime.js` for `*:changed`)

`middleware/realtime.js` (`realtimeChanges`) listens for `res.on('finish')` after any successful write to announcements, events, clubs or discussions, and broadcasts `{entity, id, action, by}`. It sends no content: clients refetch over REST.

Logging and troubleshooting

Log	Where	What it tells you
Terminal (`morgan` in dev)	`npm run dev` window	Every request: method, URL, status, time
`[error]` lines	`errorHandler`	Full stack trace of every 500
`[db]`, `[env]`, `[socket]`, `[gate-pass]`, `[ai]` prefixes	Various	Start-up and background job problems
`Activity` collection	`utils/activity.js`	Who did what; rows are deleted after 180 days by a TTL index in `models/Activity.js`
`SecurityLog` collection	`utils/securityLog.js`, Admin, then Security & Logs	Sign-ins, sessions, OTPs, scope refusals; never passwords or tokens
`LoginHistory`	`authController.recordLogin`	Per-user sign-in attempts

How to trace a failed request

1. In the browser Network tab, note the **method, URL, status and response** `message`.
2. Search the URL path in `server/src/routes/index.js` (for `/api/gate-pass/123/hod-review`, search `'/gate-pass/:id/hod-review'`).
3. Read the middleware chain left to right. For 401, look at `protect`. For 403, look at `authorize(...)` or `confinedRoleGuard` (or CSRF, if the message says so). For 422, look at the `body(...)` rules.
4. If it reached the controller, search the response `message` text in `server/src/controllers/`. The `throw new ApiError(...)` with that exact text is where it failed.
5. For a 500, read the `[error]` stack trace in the server terminal. The first line inside `server/src/` is usually the culprit.

How to debug a backend exception

1. Reproduce it with the smallest request. Use the browser, `curl`, or a test in `server/tests/` (they have helpers: `ctx.createUser`, `ctx.loginWeb`, `ctx.request`).
2. Read the stack trace bottom-up until you reach VEXON code.
3. Add a temporary `console.log({ variable })` just before the failing line, then restart (`nodemon` restarts automatically).
4. Check the data. Is a field `undefined` because the document lacks it, or because `.select()` excluded it?
5. Fix it, then **add a test** that fails before the fix and passes after (Guide 07).
6. Remove temporary logs.

You can also run `node --inspect src/server.js` and attach Chrome DevTools (open `chrome://inspect`) to set breakpoints.

How to add a small API endpoint safely

Evidence status

The endpoint below is a **hypothetical practice example**. It does not exist in VEXON. Do it on a branch, and delete the branch afterwards if you only want the practice.

Goal: `GET /api/sos/mine`, which lets a signed-in user list their own recent SOS alerts.

1. **Branch**: `git switch -c practice/sos-mine`.
2. **Controller** in `server/src/controllers/sosController.js`:

```
js
export const myAlerts = asyncHandler(async (req, res) => {
  const items = await SOSAlert.find({ user: req.user._id }).sort({ createdAt: -1 }).limit(20).lean();
  res.json({ items });
});
```

3. **Route** in `server/src/routes/index.js`, inside the Campus SOS section, *below* `router.use(protect)` and **before** any `'/sos/:id'` route:

```
js
router.get('/sos/mine', sos.myAlerts);
```

4. **Think about scope**: the filter `{ user: req.user._id }` comes from the session, never from a query parameter. A user can only see their own alerts.
5. **Test**: add a test to `server/tests/sos.test.js` that creates two users, raises an alert as user A, and asserts that user B's `/sos/mine` is empty while user A's has one item.
6. Run `npm test --prefix server` (or only that file: `node --test tests/sos.test.js` from `server/`).
7. **Client** (optional): add `getMySos: b.query({ query: () => '/sos/mine', providesTags: ['SOSAlert'] })` to `client/src/services/api.js`.
8. Review `git diff` and commit only if you meant to keep it.

Common mistake

Registering `/sos/:id/...` before `/sos/mine`. Express matches routes in order, so `mine` would be treated as an `:id` and rejected by `idParam` with "Invalid id". Static paths go before parameter paths.

Practice exercises

Exercise

1. Which middleware returns 403 when a COE user calls `GET /api/gate-pass`? **Expected:** `confinedRoleGuard` in `server/src/middleware/coeGuard.js`. The COE's allowed list has no `/gate-pass` rule.

Exercise

2. Send `POST /api/sos` with `{"type":"party"}`. What status and message come back? **Expected:** 422, "Valid SOS type is required".

Exercise

3. Find where a duplicate email on user creation becomes a readable error. **Expected:** `errorHandler` maps `err.code === 11000` to 409 "That email is already in use".

Exercise

4. Read `server/tests/auth.test.js`, test "login never reveals whether an email exists". Explain why both messages must be equal. **Expected:** different messages would let an attacker discover which emails have accounts (user enumeration).

Exercise

5. List the timers `server.js` starts, and find the file of `startTripSweeper`. **Expected:** `server/src/services/transport/trips.js`.

Source references

`server/package.json` · `server/src/server.js` · `server/src/app.js` · `server/src/config/env.js` · `server/src/config/db.js` · `server/src/config/socket.js` · `server/src/routes/index.js` · `server/src/routes/alumniRoutes.js` · `server/src/middleware/auth.js` · `server/src/middleware/coeGuard.js` · `server/src/middleware/security.js` · `server/src/middleware/rateLimit.js` · `server/src/middleware/validate.js` · `server/src/middleware/error.js` · `server/src/middleware/realtime.js` · `server/src/controllers/sosController.js` · `server/src/controllers/uploadController.js` · `server/src/controllers/aiController.js` · `server/src/utils/http.js` · `server/src/utils/activity.js` · `server/src/utils/securityLog.js` · `server/src/utils/notify.js` · `server/src/utils/storage.js` · `server/src/utils/transaction.js` · `server/src/services/otp/otpService.js` · `server/src/services/pushService.js` · `server/src/services/transport/ingest.js` · `server/tests/helpers.js` · `server/tests/auth.test.js`